

MANUEL TECHNIQUE

Souvenirs Famille

*Guide de référence pour reprendre,
comprendre et modifier l'application —
backend Laravel et frontend Next.js.*

Version du document : 22 août 2026

Dépôt : github.com/Dramane-Tanou/souvenirs-famille

Backend : Laravel 13 (PHP 8.4) · Frontend : Next.js 16 (React 19)

01 Structure du projet

02 Mise en route en local

03 Où trouver quoi (carte des fonctionnalités)

04 Base de données et modèles

05 Authentification

06 Paiement et devises

07 Livres photo et thèmes

08 Panneau d'administration

09 Déploiement (Vercel + Railway)

10 Variables d'environnement — référence

11 Recettes courantes

01 Structure du projet

Le dépôt est un monorepo avec deux applications indépendantes, chacune déployée séparément :

```
laravel-projet/
├─ app_souvenirs_famille/      API Laravel (backend)
│   └─ app/
│       ├── Http/Controllers/Api/    Contrôleurs REST
│       │   └─ Admin/                Contrôleurs réservés à l'administration
│       ├── Models/                  Modèles Eloquent
│       ├── Services/Payments/        Un fichier par prestataire de paiement
│       └─ Support/                  Classes utilitaires (devises, thèmes, paiement)
│   ├── database/migrations/          Historique du schéma, une migration par changement
│   ├── routes/api.php                Toutes les routes de l'API
│   └─ config/                       Configuration (devises, plans, services externes)
└─ souvenirs-famille-web/          Frontend Next.js
    └─ src/
        ├── app/                    Une page = un dossier (App Router)
        ├── components/              Composants réutilisés entre plusieurs pages
        ├── context/                 État global (utilisateur connecté, notifications)
        └─ lib/                     Client API, devises, thèmes, aides diverses
```

Convention à connaître : le backend utilise les attributs PHP 8 `#[Fillable(...)]` et `#[Hidden(...)]` au-dessus de chaque modèle, à la place des propriétés classiques `$fillable` / `$hidden`. C'est équivalent, juste une syntaxe plus récente.

02 Mise en route en local

Backend

```
cd app_souvenirs_famille
composer install
cp .env.example .env
php artisan key:generate
# renseigner DB_* dans .env pour pointer vers une base MySQL locale
php artisan migrate
php artisan storage:link
php artisan serve
```

Frontend

```
cd souvenirs-famille-web
npm install
# créer .env.local avec :
# NEXT_PUBLIC_API_URL=http://localhost:8000/api
npm run dev
```

Le frontend tourne alors sur `http://localhost:3000`, le backend sur `http://localhost:8000`. Le fichier `config/cors.php` du backend autorise déjà cette origine par défaut.

Sans les clés Google/Facebook/Stripe/PayPal/CinetPay dans `.env`, ces fonctionnalités restent visibles mais renvoient un message d'erreur propre plutôt qu'un plantage — normal en local sans identifiants réels.

03 Où trouver quoi

FONCTIONNALITÉ	BACKEND	FRONTEND
Inscription / connexion	AuthController.php, PhoneAuthController.php, SocialAuthController.php	app/register, app/login, components/PhoneAuthForm.tsx
Familles & membres	FamilyController.php	app/families/[id]
Fil de souvenirs	MemoryController.php	components/MemoryCard.tsx, components/LikersModal.tsx
Livres photo	BookController.php, Support/BookThemes.php	app/families/[id]/books, lib/bookThemes.ts
Commande & paiement	OrderController.php, SubscriptionController.php, Services/Payments/	app/families/[id]/books/[bookId]/order
Devise / géolocalisation	GeoController.php, Support/GeoCurrency.php, Support/CurrencyRates.php	lib/currency.ts
Administration	Admin/AdminController.php, Admin/FamilyDeletionController.php	app/admin/page.tsx

04 Base de données et modèles

Chaque changement de schéma passe par une nouvelle migration (jamais de modification directe d'une migration déjà exécutée en production). Convention de nommage :

`database/migrations/AAAA_MM_JJ_HHMMSS_description.php` .

Ajouter un champ à un modèle existant — exemple

```
php artisan make:migration add_bio_to_users_table

// dans la migration :
Schema::table('users', function (Blueprint $table) {
    $table->text('bio')->nullable()->after('gender');
});

// puis dans app/Models/User.php, ajouter 'bio' à #[Fillable(...)]
// si le champ doit être modifiable via un formulaire utilisateur.
```

Les champs sensibles comme `is_admin` et `is_super_admin` sont volontairement **absents** de `#[Fillable]` — c'est une protection délibérée contre l'assignation de masse. Pour les modifier depuis un contrôleur autorisé, utiliser `$user->forceFill(...)->save()` , jamais `update()` .

Modèles principaux

MODÈLE	RÔLE
User	Compte utilisateur — <code>is_admin</code> / <code>is_super_admin</code> gèrent les droits
Family	Cercle familial ; <code>currentPlan()</code> renvoie <code>'free'</code> ou <code>'family'</code>
Memory / MemoryLike	Une photo du fil ; likes en relation séparée
Book / BookPage / BookMemory	Un livre généré, ses pages, et la répartition des photos par page
Order / Subscription	Achat de livre / abonnement famille — mêmes champs de paiement (<code>payment_method</code> , <code>payment_reference</code> , <code>price_cents</code> , <code>currency</code>)
FamilyDeletionRequest	Journal unique des demandes de modération (<code>type</code> = <code>family_deletion</code> ou <code>member_removal</code> , avec <code>target_user_id</code> / <code>target_user_name</code> pour le second cas), approuvées ou rejetées

`is_admin` , `is_super_admin` et `is_root_super_admin` forment la hiérarchie de droits (voir section 08). Comme pour les deux premiers, `is_root_super_admin` est absent de `#[Fillable]` — protection délibérée, seule une migration ponctuelle ou un accès direct à la base peut le poser.

05 Authentification

Trois voies d'inscription, toutes exigent désormais `first_name`, `last_name`, `birth_date`, `gender` :

- **E-mail/mot de passe** — `AuthController::register()`
- **Téléphone** — `PhoneAuthController::verify()`, code à 6 chiffres en cache 10 minutes
- **Réseau social** — `SocialAuthController::callback()` ; Google/Facebook ne fournissant pas systématiquement la date de naissance ou le genre, un nouveau compte social est redirigé vers `/complete-profile` tant qu'il manque l'un des deux (le contrôle se fait au niveau du tableau de bord frontend, pas du backend).

Ajouter un nouveau fournisseur social

1. Installer le pilote Socialite correspondant si nécessaire (`composer require socialiteproviders/...`).
2. Ajouter les identifiants dans `config/services.php` et `.env`.
3. Ajouter le nom du fournisseur dans `PROVIDER_COLUMN` de `SocialAuthController.php`, et la colonne `{provider}_id` correspondante via une migration sur `users` .
4. Étendre le `whereIn('provider', [...])` des routes `/auth/{provider}/redirect` et `/callback` dans `routes/api.php`.
5. Ajouter le bouton correspondant dans `components/SocialAuthButtons.tsx`.

06 Paiement et devises

Trois prestataires implémentent la même interface `App\Contracts\PaymentGateway` (méthodes `initiate()` et `handleWebhook()`), résolue par `PaymentGatewayFactory` selon le `payment_method` choisi.

Ajouter un nouveau prestataire

1. Créer `app/Services/Payments/NouveauGateway.php` qui implémente `PaymentGateway`.
2. L'enregistrer dans `PaymentGatewayFactory::make()`.
3. Ajouter le nom dans les règles de validation `payment_method` de `OrderController` et `SubscriptionController`, et dans `config/currencies.php` → `payment_methods` pour préciser quelles devises il accepte.
4. Toute confirmation de paiement doit in fine appeler `App\Support\PaymentReconciler::markPaid($reference)` — c'est le seul endroit qui fait passer une commande ou un abonnement à l'état payé.

Devises — couverture mondiale

`Support/GeoCurrency.php` déduit pays/devise depuis l'IP du visiteur (API gratuite `ip-api.com`, repli sur CH/CHF en local). Couverture actuelle : CHF (Suisse/Liechtenstein), EUR (zone euro), GBP (Royaume-Uni), CAD (Canada), AUD (Australie/Nouvelle-Zélande), XOF (zone UEMOA — Afrique de l'Ouest), XAF (zone CEMAC — Afrique centrale), et **USD en repli universel** pour tout le reste du monde — aucun visiteur ne se retrouve donc sans devise utilisable. `Support/CurrencyRates.php` récupère des taux réels une fois par jour (`open.er-api.com`), avec repli sur les taux fixes de `config/currencies.php` si l'appel échoue.

Ajouter une devise supplémentaire

1. Ajouter le taux de repli, le symbole et (si zéro décimale) l'entrée dans `config/currencies.php`.
2. Ajouter le ou les pays correspondants dans `Support/GeoCurrency.php` → `currencyForCountry()`.
3. Reporter les mêmes valeurs dans `src/lib/currency.ts` (frontend) — les sélecteurs de devise sur les pages de commande/abonnement sont dynamiques et n'ont besoin d'aucune autre modification.

Mode test des paiements

`config/payments.php` → `test_mode` (variable `PAYMENTS_TEST_MODE`). Actif, il court-circuite `PaymentGatewayFactory` dans `OrderController::store()` et `SubscriptionController::upgrade()` : la commande/l'abonnement est marqué payé immédiatement, sans appeler de vrai prestataire. Permet de tester tout le parcours (y compris le téléchargement du livre PDF) avant d'avoir des comptes marchands réels.

À désactiver (`PAYMENTS_TEST_MODE=false`) dès que Stripe/PayPal/CinetPay sont réellement configurés — sinon aucune commande ne sera facturée pour de vrai.

07 Livres photo et thèmes

Les thèmes sont définis à deux endroits qui doivent rester synchronisés à la main :

- Backend — `app/Support/BookThemes.php` (utilisé pour générer le PDF via dompdf)
- Frontend — `src/lib/bookThemes.ts` (utilisé pour l'aperçu à l'écran)

Les thèmes sont volontairement **CSS pur** (pas d'image ni de police externe) pour garantir un rendu identique entre l'aperçu web et le PDF généré par dompdf, qui ne supporte pas tout le CSS moderne.

Ajouter un thème

Dupliquer une entrée existante dans les deux fichiers avec un nouvel identifiant, ajuster les couleurs/polices, puis vérifier visuellement l'aperçu **et** télécharger un PDF de test pour confirmer que le rendu correspond.

08 Panneau d'administration

Trois niveaux d'accès, portés par trois colonnes booléennes sur `users` : `is_admin` , `is_super_admin` , `is_root_super_admin` . Le middleware `EnsureIsAdmin` accepte `is_admin` ou `is_super_admin` ; `EnsureIsSuperAdmin` n'accepte que `is_super_admin` . Toutes les routes sont préfixées `/api/admin` dans `routes/api.php`.

Hiérarchie des droits

- **Admin** — accès au tableau de bord, gestion des familles, peut *demande* une suppression de famille ou un retrait de membre (jamais l'exécuter directement).
- **Super-admin** — tout ce qu'un admin peut faire, plus : nommer/rétrograder/retraiter d'autres admins et super-admins, approuver ou rejeter les demandes, agir directement (suppression de famille, retrait de membre) sans approbation.
- **Super-admin racine** (`is_root_super_admin`) — un seul compte, jamais modifiable par personne d'autre. Posé uniquement en base de données ou par migration, jamais via une route applicative (voir `AdminController::demoteAdmin()` et `demoteSuperAdmin()` , qui vérifient systématiquement `abort_if($user->is_root_super_admin, ...)`).

Suppressions et retraits : demande + approbation

`Admin/FamilyDeletionController.php` centralise les deux types d'action destructive sur une famille, dans la table `family_deletion_requests` :

- `requestDeletion()` / `requestMemberRemoval()` — un admin simple crée une demande `pending` , avec raison obligatoire.
- `approve()` / `reject()` — réservés au super-admin ; l'approbation exécute réellement l'action (branchement sur `type`).
- `destroyDirect()` / `removeMemberDirect()` — le super-admin agit sans passer par une demande ; l'action est tout de même journalisée dans la même table, déjà au statut `approved` , pour garder un historique complet.
- `myRequests()` — permet à l'admin qui a soumis une demande de voir la décision du super-admin (onglet « *Mes demandes* » côté frontend), sans avoir accès à l'historique complet des autres.

Ajouter une nouvelle métrique au tableau de bord

1. Calculer la valeur dans `AdminController::overview()` et l'ajouter au tableau JSON renvoyé.
2. Ajouter l'entrée correspondante dans le tableau `statCards` de `app/admin/page.tsx` (frontend).

Nommer le tout premier super-administrateur racine

Il n'existe volontairement aucune route pour le faire depuis l'application. Deux options : un accès direct à la base de données, ou une migration ponctuelle comme celle utilisée pour ce lancement (`2026_08_22_200000_add_is_root_super_admin_to_users_table.php`) — à utiliser comme modèle si un

successeur doit un jour être désigné. Une fois le premier super-admin racine posé, tous les suivants (super-admins ou admins simples) peuvent être nommés normalement depuis l'onglet *Administrateurs*.

09 Déploiement (Vercel + Railway)

Frontend — Vercel

```
cd souvenirs-famille-web
vercel --prod
```

Le déploiement se fait aussi automatiquement à chaque `git push` sur `main` une fois le dépôt GitHub lié dans le tableau de bord Vercel.

Backend — Railway

```
cd app_souvenirs_famille
railway up --service backend
```

Le Procfile exécute automatiquement `php artisan migrate --force` puis `storage:link` avant de démarrer le serveur — toute nouvelle migration poussée est donc appliquée au déploiement suivant, sans étape manuelle.

Les photos envoyées par les utilisateurs sont stockées sur un **volume Railway persistant**, monté sur `/app/storage/app/public`. Sans ce volume, tout fichier envoyé serait perdu au déploiement suivant — ne jamais le détacher du service `backend`.

10 Variables d'environnement — référence

VARIABLE	RÔLE
APP_KEY	Clé de chiffrement Laravel — générée une fois, ne jamais changer en production sans re-chiffrer les données concernées
DB_*	Connexion MySQL
FRONTEND_URL	Origine autorisée en CORS côté backend — doit correspondre exactement à l'URL Vercel
NEXT_PUBLIC_API_URL	URL de l'API que le frontend appelle — doit inclure <code>/api</code>
GOOGLE_*, FACEBOOK_*, APPLE_*	Identifiants OAuth — vides = bouton correspondant désactivé proprement
STRIPE_*, PAYPAL_*, CINETPAY_*	Identifiants de paiement — vides = moyen de paiement correspondant refusé avec message clair
PAYMENTS_TEST_MODE	<code>true</code> = commandes et abonnements validés instantanément sans prestataire réel. À repasser à <code>false</code> avant tout usage avec de vrais paiements.

11 Recettes courantes

Ajouter une nouvelle route API protégée

```
// routes/api.php, dans le groupe Route::middleware('auth:sanctum')
Route::get('/mon-endpoint', [MonController::class, 'methode']);
```

Voir les erreurs backend en production

```
railway logs --service backend
```

Réinitialiser la base de données locale

```
php artisan migrate:fresh
```

Ne jamais exécuter `migrate:fresh` contre la base de production — elle supprime toutes les tables avant de les recréer.

Tester les paiements sans clés réelles

Sans identifiants Stripe/PayPal/CinetPay renseignés et `PAYMENTS_TEST_MODE=false`, `PaymentGatewayNotConfigured` est levée et interceptée pour renvoyer une erreur 422 propre. Avec `PAYMENTS_TEST_MODE=true`, aucun prestataire n'est appelé du tout et le paiement est validé instantanément (voir section 06).

Se connecter en SSH à un service Railway (ex. lire les logs d'erreur applicatifs)

```
railway ssh --service backend -- tail -n 80 storage/logs/laravel.log
```

Les logs de requêtes (`railway logs`) ne contiennent pas la pile d'erreur PHP complète. Pour ça, il faut lire `storage/logs/laravel.log` directement dans le conteneur via SSH. Deux prérequis, à faire une seule fois par machine :

```
# 1. Enregistrer sa clé publique auprès de Railway
railway ssh keys add

# 2. Faire confiance à l'hôte SSH de Railway (sinon "Host key verification failed")
ssh-keyscan -H ssh.railway.com >> ~/.ssh/known_hosts
```